Практична робота №7

Експерименти з підвищеним навантаженням: оптимізація серверних скриптів

Тривалість: 90 хв

Формат: комп'ютерний практикум

Мета роботи

Навчитися вимірювати продуктивність власного PHP-застосунку (час виконання, використання пам'яті), проводити просте навантажувальне тестування засобом Apache Bench (`ab`) і застосовувати базові прийоми оптимізації - кешування результатів запитів, індекси бази даних, зменшення кількості SQL-запитів на один HTTP-запит.

Необхідні знання й інструменти

Знання: виконані практикуми №1–6 (PDO/MySQL, власний API з практикуму №6); базове розуміння SQL-запитів і структури таблиці вашого домену.

Інструменти: редактор коду; локальний вебсервер із PHP і MySQL/MariaDB; `ab` (Apache Bench) — постачається разом з Apache (XAMPP: `xampp\apache\bin\ab.exe`), у Docker-образах httpd або окремим пакетом `apache2-utils` на Linux; phpMyAdmin або консольний клієнт `mysql` для команди `EXPLAIN`.

Короткий теоретичний вступ

Профілювання - вимірювання часу виконання й пам'яті безпосередньо в коді, без сторонніх інструментів: `microtime(true)` до і після ділянки коду дає час у секундах, `memory_get_peak_usage(true)` - пікове використання пам'яті процесом:

```
$start = microtime(true);
// ... код, що вимірюється (наприклад, listBooks($pdo)) ...
$elapsed = (microtime(true) - $start) * 1000; // у мілісекундах
$peakMemory = memory_get_peak_usage(true) / 1024 / 1024; // у мегабайтах
error_log(sprintf('Час: %.2f мс, пам'ять: %.2f МБ', $elapsed, $peakMemory));
```

Навантажувальне тестування - на відміну від профілювання одного виконання, `ab` імітує багато паралельних клієнтів і показує, як застосунок поводиться під навантаженням:

```
ab -n 200 -c 20 "http://localhost/api.php?resource=books"
```

Тут `-n 200` - загальна кількість запитів, `-c 20` - кількість одночасних («паралельних») клієнтів. У виведенні найважливіші рядки: `Requests per second` (скільки запитів застосунок витримує за секунду), `Time per request` (середній час одного запиту, є два рядки - для одного запиту і в

перерахунку на паралельність) і `Failed requests` (кількість запитів, що завершилися помилкою чи не отримали відповіді - у нормі має бути 0).

Проблема N+1 запитів - типова причина повільного відгуку: замість одного запиту, що одразу повертає потрібні агреговані чи пов'язані дані, код виконує окремий SQL-запит для кожного рядка результату в циклі (1 запит на список + N запитів у циклі замість 1+1):

```
// ПОВІЛЬНО: N+1 — окремий запит для кожного запису в циклі
$books = $pdo->query('SELECT * FROM books')->fetchAll();
foreach ($books as &$book) {
    $stmt = $pdo->prepare('SELECT COUNT(*) FROM books WHERE author = :author');
    $stmt->execute([':author' => $book['author']]);
    $book['author_book_count'] = $stmt->fetchColumn();
}

// ШВИДКО: один запит із GROUP BY замість запиту в циклі
$counts = $pdo->query(
    'SELECT author, COUNT(*) AS cnt FROM books GROUP BY author'
)->fetchAll(PDO::FETCH_KEY_PAIR);
```

Індекси бази даних - прискорюють пошук і фільтрацію за стовпцем ціною повільнішого запису й додаткового місця на диску; без індексу СУБД перебирає всі рядки таблиці (повне сканування). Команда `EXPLAIN` перед SQL-запитом показує план виконання - стовпець `type` (наприклад, `ALL` - повне сканування, `ref`/`range` - використано індекс) і `rows` (приблизна кількість рядків, які СУБД переглядає):

```
EXPLAIN SELECT * FROM books WHERE author = 'Іван Франко';
-- до індексу: type=ALL, rows≈<кількість рядків у таблиці>
CREATE INDEX idx_books_author ON books (author);
EXPLAIN SELECT * FROM books WHERE author = 'Іван Франко';
-- після індексу: type=ref, rows≈<кількість книг цього автора>
```

Кешування результатів запитів - якщо результат важкого запиту (агрегат, рідко змінюваний список) потрібен часто, а сам він змінюється рідко, його можна зберегти на деякий час і повертати без звернення до БД. Найпростіший варіант - файловий кеш із часом життя (TTL):

```
function cachedQuery(PDO $pdo, string $key, string $sql, int $ttlSeconds = 60): mixed
{
    $file = sys_get_temp_dir() . '/cache_' . md5($key) . '.json';
    if (is_file($file) && (time() - filemtime($file)) < $ttlSeconds) {
        return json_decode(file_get_contents($file), true);
    }
    $result = $pdo->query($sql)->fetchAll();
    file_put_contents($file, json_encode($result));
    return $result;
```

```
}
```

За наявності розширення APCu (`apcu_fetch()`/`apcu_store()`) кеш можна тримати в пам'яті сервера замість файлів - це швидше, але дані пропадають при перезапуску PHP-процесу; для практикуму достатньо файлового кешу або APCu, обидва варіанти прийнятні.

Хід роботи

Структура завдання спільна для всіх варіантів - відрізняється лише ендпоінт, стовпець для індексу й сценарій N+1/кешування (див. розділ «20 варіантів завдання»). Виконуйте кроки послідовно.

Крок 1. Виміряти базовий час виконання «як є».

Обгорніть виклик вашого спискового API-ендпоінта (`api.php?resource=...`) з практикуму №6 викликами `microtime(true)` і `memory_get_peak_usage(true)` до і після; запишіть отримані значення - це ваша базова лінія («до»).

Крок 2. Провести навантажувальне тестування «як є».

Запустіть `ab -n 200 -c 20` на цей самий ендпоінт; зафіксуйте `Requests per second`, `Time per request` (mean) і `Failed requests` - це також частина базової лінії.

Крок 3. Знайти вузьке місце - порахувати кількість SQL-запитів.

Додайте лічильник викликів `query()`/`execute()` (наприклад, статичну змінну або обгортку над PDO) і виведіть, скільки SQL-запитів виконується на один виклик вашого ендпоінта; для сценарію вашого варіанта (розділ «20 варіантів завдання») це має показати проблему N+1.

Крок 4. Додати індекс на стовпець фільтрації вашого домену.

Виконайте `EXPLAIN` для запиту з фільтром вашого варіанта до створення індексу, створіть `CREATE INDEX ...` на потрібний стовпець, повторіть `EXPLAIN` і порівняйте `type`/`rows` до і після.

Крок 5. Усунути N+1-запит.

Перепишіть сценарій вашого варіанта так, щоб замість запиту в циклі виконувався один запит (`GROUP BY`, `JOIN` або одне агрегатне `SELECT`) - переконайтеся лічильником із кроку 3, що кількість запитів справді зменшилася.

Крок 6. Додати кешування «важкого» запиту вашого варіанта.

Реалізуйте файловий кеш або APCu для агрегатного/спискового запиту вашого домену з розумним TTL (наприклад, 30–60 секунд); переконайтеся, що після зміни даних (додавання/редагування запису) кеш або інвалідується явно, або природно застаріває за TTL, а не показує назавжди застарілі дані.

Крок 7. Повторно виміряти показники з кроків 1–2 після оптимізацій.

Виконайте той самий профайлінг і той самий `ab`-тест на оптимізованому коді; зафіксуйте нові значення часу, пам'яті, `Requests per second`.

Крок 8. Оформити короткий звіт «до / після».

У README чи коментарі на початку скрипта зведіть у таблицю чи список: час виконання, кількість SQL-запитів, `Requests per second` - до оптимізацій і після; додайте одне речення пояснення причини різниці.

20 варіантів завдання

Наскрізні варіанти - той самий номер, що й у попередніх практикумах, відповідає одному домену протягом усього курсу. Виберіть свій варіант і виконуйте завдання для нього.

№ / Домен	Зміст варіанта
1. Бібліотека (каталог книг, видача/повернення)	**Навантаження:** списковий ендпоінт `resource=books` (усі книги). **Індекс:** стовпець `author` (використовується у `findByAuthor()`). **N+1:** для кожної книги окремий запит «скільки ще книг цього автора» замість одного запиту `SELECT author, COUNT(*) FROM books GROUP BY author`. **Кеш:** список доступних книг `listAvailable()` - змінюється нечасто, кешувати на 30–60 с.
2. Інтернет-магазин (каталог товарів, кошик)	**Навантаження:** списковий ендпоінт `resource=products`. **Індекс:** стовпець `sku` (використовується у `findBySku()`). **N+1:** для кожного товару окремий запит суми `price * stock` замість агрегатного `totalStockValue()` в один запит. **Кеш:** результат `totalStockValue()` - важкий агрегат по всьому каталогу.
3. Блог/CMS для статей	**Навантаження:** списковий ендпоінт `resource=articles`. **Індекс:** стовпець `author` (використовується у `findByAuthor()`). **N+1:** для кожної статті окремий запит кількості статей того самого автора замість `GROUP BY author`. **Кеш:** `listRecent($limit)` - список останніх статей на головній сторінці.
4. Фітнес-трекер (тренування, вправи)	**Навантаження:** списковий ендпоінт `resource=workouts`. **Індекс:** стовпець `type`. **N+1:** для кожного тренування окремий запит сумарних калорій цього типу замість `totalCalories()` з `GROUP BY type` в один запит. **Кеш:** результат `totalCalories()` за поточний фільтр.
5. Кінотека (фільми, рецензії)	**Навантаження:** списковий ендпоінт `resource=movies`. **Індекс:** стовпець `genre`. **N+1:** для кожного фільму окремий запит середнього рейтингу жанру замість одного

	агрегатного запиту. **Кеш:** `topRated($limit)` - список найкращих фільмів.
6. Кулінарна книга (рецепти, інгредієнти)	**Навантаження:** списковий ендпоінт `resource=recipes`. **Індекс:** стовпець `cook_time_min`. **N+1:** для кожного рецепта окремий запит кількості інших рецептів із таким самим інгредієнтом замість одного запиту з `LIKE`. **Кеш:** `shortestCookTime()` - список найшвидших рецептів.
7. Трекер завдань (to-do list, проєкти)	**Навантаження:** списковий ендпоінт `resource=tasks`. **Індекс:** стовпець `priority`. **N+1:** для кожного завдання окремий запит кількості завдань з таким самим `priority` замість `GROUP BY priority`. **Кеш:** `pendingTasks()` - список невиконаних завдань.
8. Система бронювання (готель/зали)	**Навантаження:** списковий ендпоінт `resource=rooms`. **Індекс:** стовпець `capacity`. **N+1:** для кожної кімнати окремий запит кількості кімнат із такою самою місткістю замість одного агрегатного запиту. **Кеш:** `findAvailable($capacity)` - список вільних кімнат за місткістю.
9. Форум/дошка обговорень	**Навантаження:** списковий ендпоінт `resource=topics`. **Індекс:** стовпець `author`. **N+1:** для кожної теми окремий запит кількості тем того самого автора замість `GROUP BY author`. **Кеш:** `listPinned()` - список закріплених тем.
10. Каталог вакансій (job board)	**Навантаження:** списковий ендпоінт `resource=vacancies`. **Індекс:** стовпець `company`. **N+1:** для кожної вакансії окремий запит середньої зарплати цієї компанії замість одного агрегатного запиту. **Кеш:** `avgSalary()` - середня зарплата по всьому каталогу.
11. Система обліку студентів/оцінок	**Навантаження:** списковий ендпоінт `resource=students`. **Індекс:** стовпець `group_name`. **N+1:** для кожного студента окремий запит середнього балу його групи замість `GROUP BY group_name`. **Кеш:** `topStudent()` - студент із найвищим середнім балом.
12. Онлайн-опитування/анкети	**Навантаження:** списковий ендпоінт `resource=questions`. **Індекс:** стовпець `type`. **N+1:** для кожного питання окремий запит

	кількості питань цього типу замість `countByType()` в один запит. **Кеш:** результат `countByType()`.
13. Трекер особистих витрат (фінанси)	**Навантаження:** списковий ендпоінт `resource=transactions`. **Індекс:** стовпець `category`. **N+1:** для кожної транзакції окремий запит суми по цій категорії замість `totalByCategory()` в один запит. **Кеш:** `balance()` - поточний баланс.
14. Каталог музики/плейлисти	**Навантаження:** списковий ендпоінт `resource=tracks`. **Індекс:** стовпець `artist`. **N+1:** для кожного треку окремий запит сумарної тривалості цього виконавця замість `GROUP BY artist`. **Кеш:** результат `totalDuration()`.
15. Система бронювання квитків (кіно/події)	**Навантаження:** списковий ендпоінт `resource=events`. **Індекс:** стовпець `venue`. **N+1:** для кожної події окремий запит кількості подій у тому самому `venue` замість `GROUP BY venue`. **Кеш:** `soldOutList()` - список подій без вільних місць.
16. Дошка оголошень	**Навантаження:** списковий ендпоінт `resource=listings`. **Індекс:** стовпець `category`. **N+1:** для кожного оголошення окремий запит найдешевшого оголошення в тій самій категорії замість одного запиту з `MIN(price)` за `category`. **Кеш:** `cheapest()` - найдешевше оголошення.
17. Трекер звичок (habit tracker)	**Навантаження:** списковий ендпоінт `resource=habits`. **Індекс:** стовпець `current_streak` (сортування рейтингу серій). **N+1:** для кожної звички окремий запит середнього `current_streak` усіх звичок замість одного агрегатного запиту. **Кеш:** повний список звичок `listAll()`.
18. Каталог догляду за рослинами (садівництво)	**Навантаження:** списковий ендпоінт `resource=plants`. **Індекс:** стовпець `watering_interval_days`. **N+1:** для кожної рослини окремий запит кількості рослин із таким самим інтервалом поливу замість `GROUP BY watering_interval_days`. **Кеш:** `needsWaterToday()` - список рослин, що потребують поливу.
19. Система тікетів служби підтримки (helpdesk)	**Навантаження:** списковий ендпоінт `resource=tickets`. **Індекс:** стовпець `status`. **N+1:** для кожного тікета окремий запит кількості тікетів з таким самим `priority` замість `GROUP BY priority`.

	Кеш: `openTickets()` - список відкритих тікетів.
20. Спортивна ліга (турнірна таблиця, матчі, команди)	**Навантаження:** списковий ендпоінт `resource=teams`. **Індекс:** стовпець `city`. **N+1:** для кожної команди окремий запит середніх очок команд того самого міста замість `GROUP BY city`. **Кеш:** `standings()` - турнірна таблиця.